

Digital Logic and Digital System

JAHEZIA

**College of Science
Northern Border University**



FIVE GENERATION OF COMPUTERS

First Generation (1940 – 1956): Vacuum Tubes

Technology Used: Vacuum tubes for circuitry and magnetic drums for memory.

Characteristics:

Very large in size, consumed a lot of power, and generated excessive heat.

Machine language (binary) was used for programming.

Input/output through punched cards and paper tape.

Limited speed and reliability.

Examples:

- ENIAC (Electronic Numerical Integrator and Computer)
- UNIVAC I (Universal Automatic Computer)

Applications:

- Scientific calculations
- Military purposes (ballistic calculations)



Second Generation (1956 – 1963): Transistors

Technology Used: **Transistors replaced vacuum tubes.**

Characteristics:

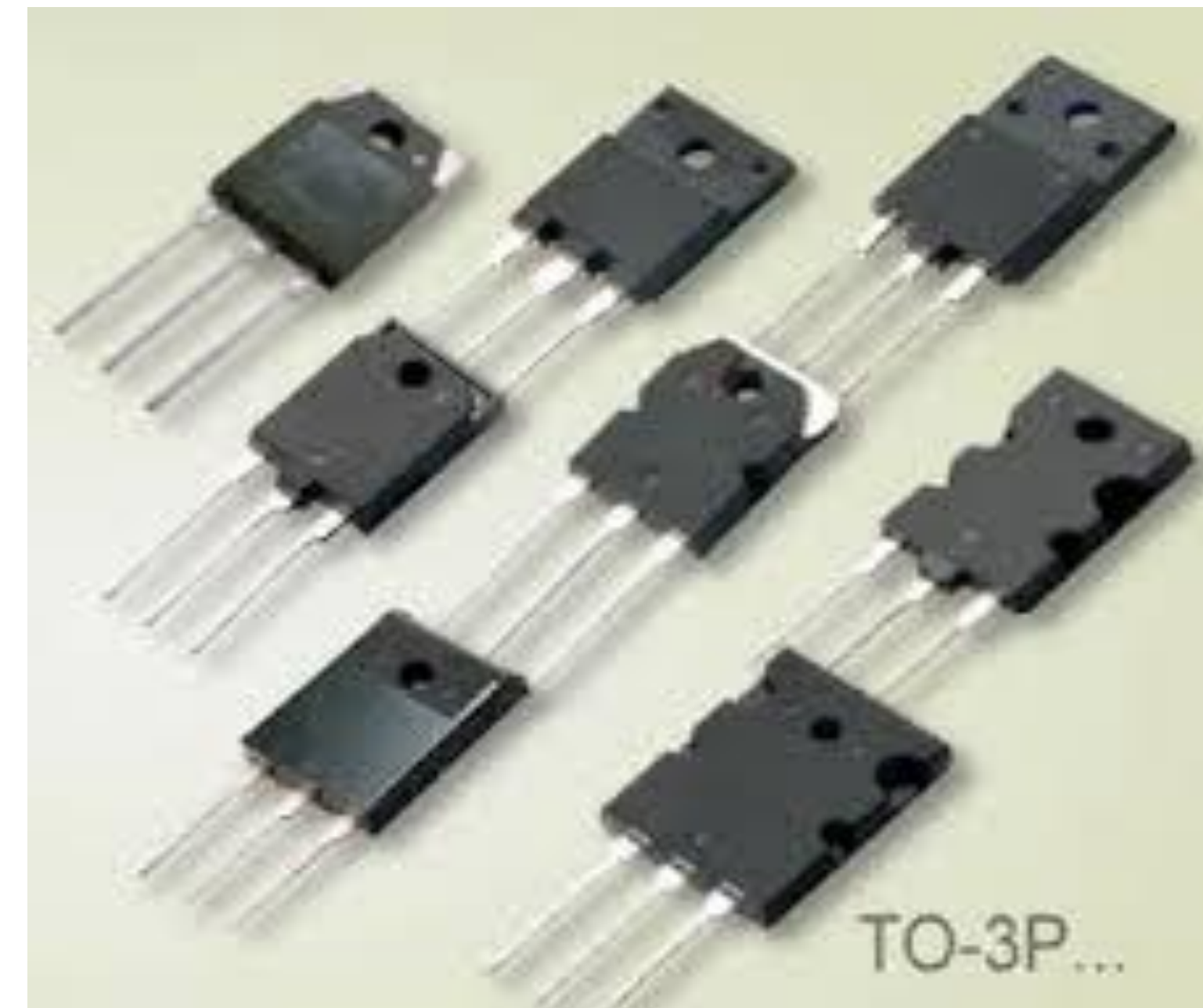
- Smaller, faster, more reliable, and energy-efficient than first-generation computers.
- Used magnetic core memory.
- Programming done in assembly language and early high-level languages (COBOL, FORTRAN).

Examples:

- IBM 7094
- UNIVAC II

Applications:

- Business data processing
- Scientific and engineering calculations



Third Generation (1964 – 1971): Integrated Circuits (ICs)

Technology Used: **Integrated Circuits** (multiple transistors on a single silicon chip).

Characteristics:

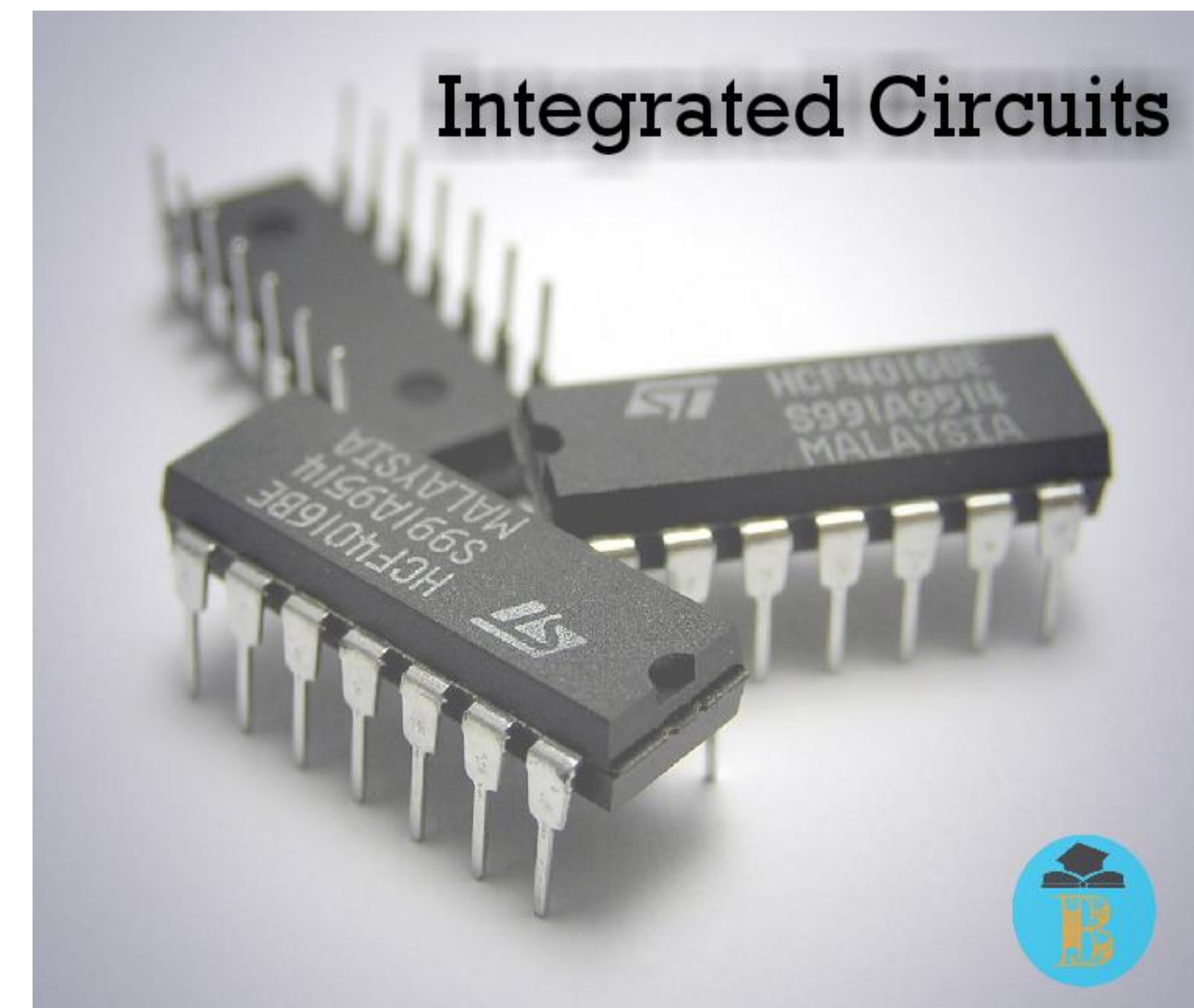
- Much smaller and more powerful.
- Used keyboards, monitors, and operating systems.
- Supported multitasking and multiprocessing.

Examples:

- IBM 360 series
- PDP-8 and PDP-11

Applications:

- Commercial and scientific applications
- General-purpose computing



Fourth Generation (1971 – 1980s): Microprocessors

Technology Used: **Microprocessors** — thousands of integrated circuits built into a single silicon chip.

Characteristics:

- Personal computers (PCs) were introduced.
- Use of Very Large-Scale Integration (VLSI).
- Graphical User Interfaces (GUIs), networks, and portable computers emerged.

Examples:

- Intel 4004, 8085, 8086 processors
- Apple II, IBM PC

Applications:

Word processing, databases, spreadsheets
Business, education, and home use



Fifth Generation (1980s – Present): Artificial Intelligence (AI) and Beyond

Technology Used: Based on Artificial Intelligence (AI), machine learning, and **parallel processing**.

Characteristics:

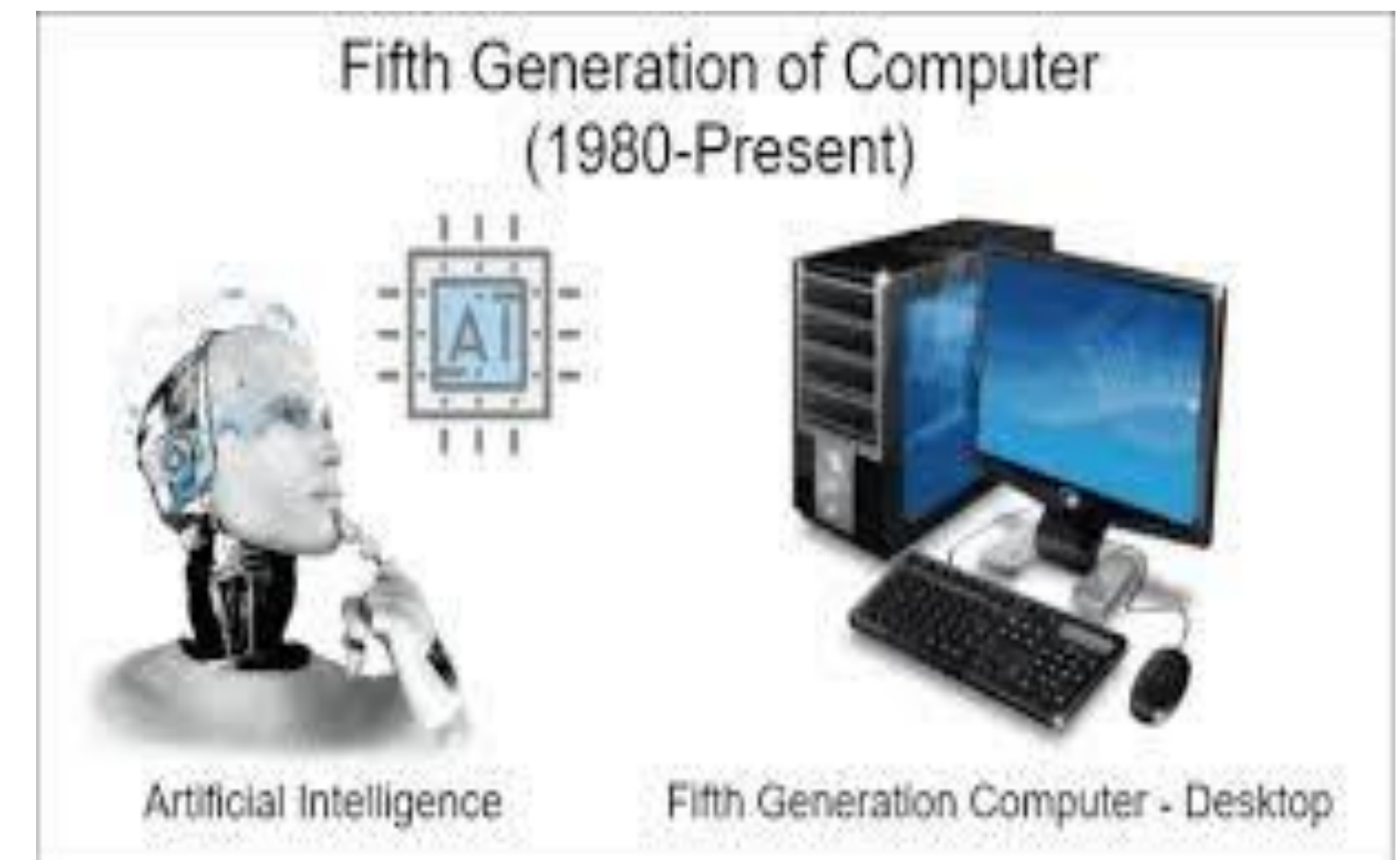
- Use of Very Large-Scale Integration (VLSI) and Ultra Large-Scale Integration (ULSI).
- Natural Language Processing (NLP), robotics, expert systems.
- Quantum computation and cloud computing emerging.

Examples:

- IBM Watson
- Google DeepMind
- Modern laptops, smartphones, AI-driven systems

Applications:

- Expert systems, robotics, voice recognition, machine learning, automation.



2. Multi-core architecture



Overview:

A multi-core processor contains two or more independent processing units (cores) on a single chip. Each core can execute instructions independently, improving performance and efficiency.

Key Features:

- **Parallel Processing:** Multiple tasks run simultaneously across cores.
- **Shared Cache and Memory:** Cores share system memory but may have private or shared cache levels.
- **Energy Efficiency:** Multiple cores running at lower frequencies consume less power than one high-speed core.
- **Scalability:** Performance scales with software designed for parallelism (multi-threaded programs).

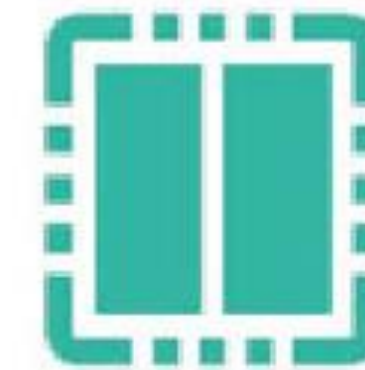
2. Multi-core architecture

Advantages:

- Faster execution of multitasking and computational tasks.
- Better system responsiveness.
- Improved performance for modern applications like AI, gaming, and video processing.

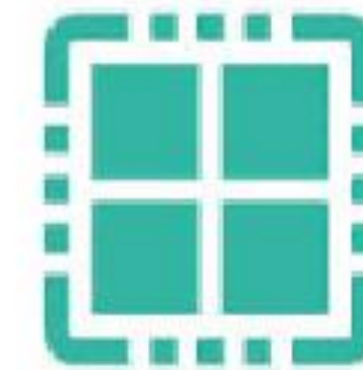
Examples of Multi-core CPUs:

- Dual-core: 2 cores (e.g., Intel Core 2 Duo)
- Quad-core: 4 cores (e.g., Intel i5, AMD Ryzen 5)
- Octa-core: 8 cores (e.g., smartphone processors)



Dual Core

Suitable for basic use. May struggle to multi-task and run more demanding apps.



Quad Core

Suitable for everyday tasks like social media, listening to music and browsing the web.



Octa Core

Great for gaming and managing several tabs and apps at once.

3. Different Representations of Computer Functionality, Including Logical Expressions and Gates



Overview:

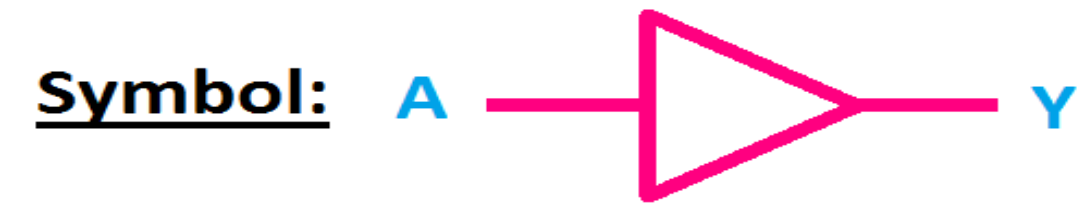
Computer functionality at the **hardware level is based on digital logic** — the manipulation of binary data using logical operations. These operations can be represented in multiple forms to describe how computers make decisions and perform calculations.

- At the core of every computer lies digital logic, which enables the computer to process binary information (0s and 1s).
- Every operation a computer performs—whether arithmetic, comparison, or decision-making—is based on logical operations.
- To understand how computers work at the machine level, we use different representations such as logical expressions, truth tables, and logic gate diagrams.

Figure. 01

Basic Logic Gates with Truth Table

YES Gate



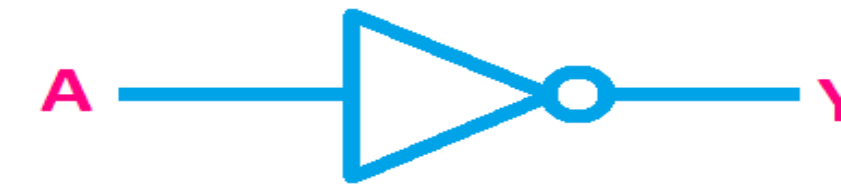
Boolean Expression: $Y = A$

Truth Table:

INPUT A	OUTPUT (Y)
0	0
1	1

NOT Gate

WWW.ETechnoG.COM



$$Y = A'$$

INPUT A	OUTPUT (Y)
0	1
1	0

AND Gate



$$Y = A \cdot B$$

INPUT		OUTPUT (Y)
A	B	
0	0	1
0	1	1
1	0	1
1	1	0

NAND Gate



$$Y = \overline{A \cdot B}$$

OR Gate



$$Y = A + B$$

INPUT		OUTPUT (Y)
A	B	
0	0	0
0	1	1
1	0	1
1	1	1

NOR Gate



$$Y = \overline{A + B}$$

INPUT		OUTPUT (Y)
A	B	
0	0	1
0	1	0
1	0	0
1	1	0

XOR Gate



$$Y = A \oplus B$$

INPUT		OUTPUT (Y)
A	B	
0	0	1
0	1	0
1	0	0
1	1	1

XNOR Gate



$$Y = \overline{A \oplus B}$$

4. Basic Building Blocks of a Computer: Arithmetic-Logic Unit, Registers, Central Processing Unit, and Memory

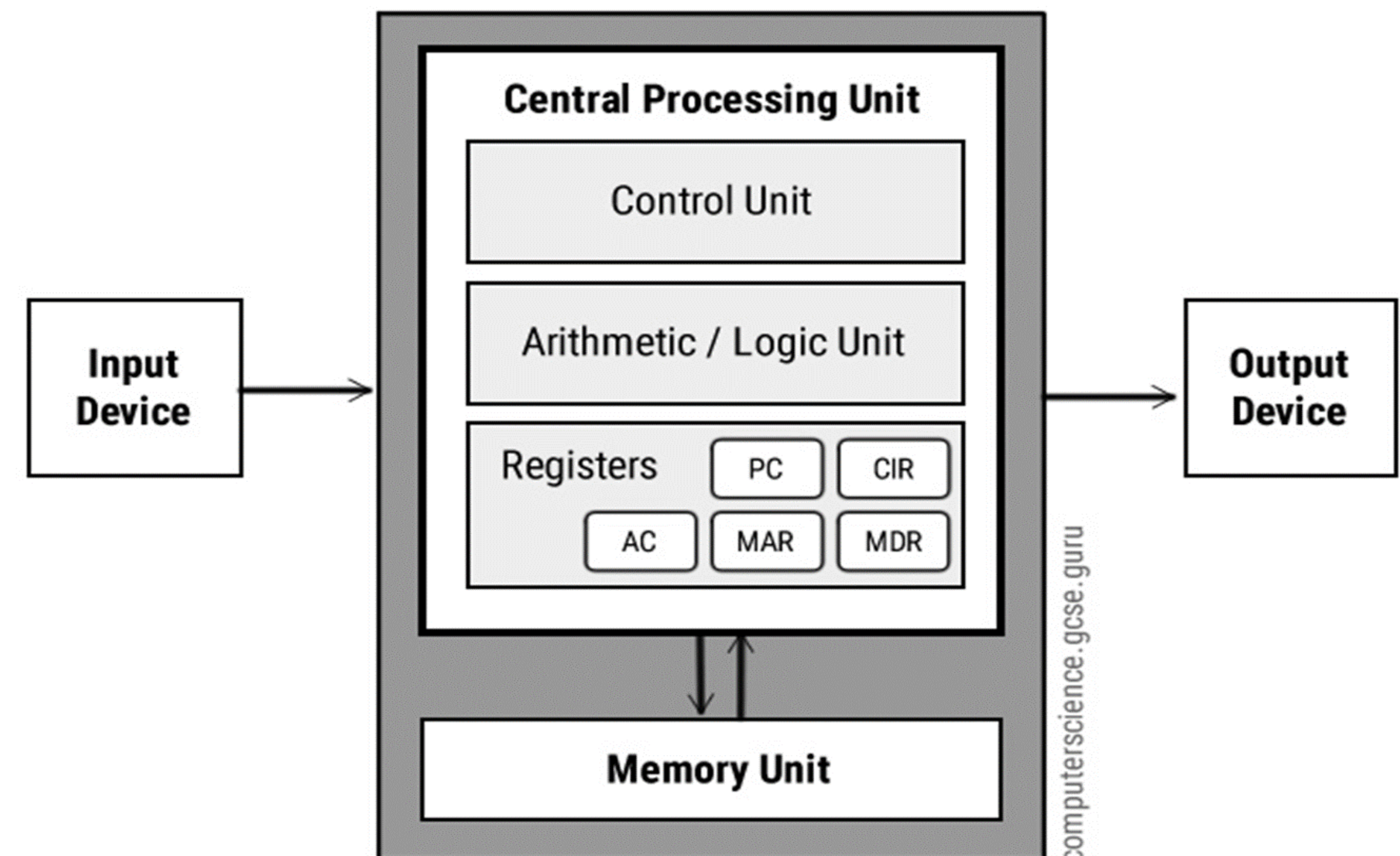
Overview

A computer is made up of several core components that work together to process data and execute instructions.

These components form the computer architecture, and they include:

1. Arithmetic Logic Unit (ALU)
2. Registers
3. Central Processing Unit (CPU)
4. Memory

Each plays a specific role in the fetch-decode-execute cycle of processing.



1. Arithmetic Logic Unit (ALU)

Definition:

The ALU is the part of the CPU that performs all arithmetic (mathematical) and logical (decision-making) operations.

Functions:

- Arithmetic Operations:
Addition, subtraction, multiplication, division.
- Logical Operations:
Comparisons such as AND, OR, NOT, XOR, greater than, less than, equal to.
- Bitwise Operations:
Shifting bits left or right.

Working Example:

If a computer needs to calculate $A + B$,

- The CPU sends both A and B to the ALU.
- The ALU performs the addition.
- The result is stored in a register or memory.

Importance:

The ALU acts as the “mathematical brain” of the computer.

2. Registers

Definition:

Registers are small, high-speed storage units located inside the CPU. They temporarily hold data, instructions, or intermediate results that the CPU is currently processing.

Characteristics:

- Faster than main memory (RAM).
- Limited in number and size (typically 8, 16, 32, or 64 bits wide).
- Used directly by the processor during execution.

Types of Registers:

- Accumulator (ACC): Holds results of ALU operations.
- Program Counter (PC): Holds the address of the next instruction.
- Instruction Register (IR): Holds the current instruction being executed.
- Memory Address Register (MAR): Stores the address of the memory location to access.
- Memory Data Register (MDR): Holds data read from or written to memory.
- Status Register (Flag Register): Stores information about the result of an operation (e.g., zero, carry, overflow).

Importance:

Registers allow the CPU to work quickly by keeping essential data close to the processing unit.

3. Central Processing Unit (CPU)

Definition:

The Central Processing Unit is the main control unit of the computer — often called the “brain” of the computer.

It performs all processing tasks by coordinating between input, memory, and output.

Major Components of CPU:

- Arithmetic Logic Unit (ALU) – Performs calculations and logic operations.
- Control Unit (CU) – Directs the flow of data and instructions within the computer.
- Registers – Temporarily store data and instructions for quick access.

Functions of CPU:

- Fetch: Retrieves instructions from memory.
- Decode: Interprets the instruction.
- Execute: Performs the required operation via the ALU.
- Store: Saves the result back into memory or a register.

Importance:

The CPU manages and executes all tasks that make the computer function.

4.Memory

Definition:

Memory is used to store data and instructions needed by the CPU.
It can be divided into primary (main) memory and secondary (auxiliary) memory.

Types of Memory:

1.Primary Memory (Main Memory):

- RAM (Random Access Memory): Temporary, volatile memory used while the computer is running.
- ROM (Read-Only Memory): Permanent memory containing essential programs (like BIOS).

2.Secondary Memory:

- Includes hard drives, SSDs, USBs — used for long-term data storage.

Characteristics:

- Measured in bytes, kilobytes, megabytes, gigabytes, etc.
- Provides data to CPU for processing and stores results.

Importance:

Without memory, a computer cannot store data or run programs.

Simplified Working Example:

1. The CPU fetches an instruction from memory (e.g., “add A and B”).
2. The Control Unit decodes it.
3. The ALU performs the addition using data stored in registers.
4. The result is stored back into a register or memory.

This process repeats millions of times per second!

6. CAD Simulation Tools for Evaluating Simple Building Blocks of a Computer

Overview:

CAD (Computer-Aided Design) simulation tools are software applications used to design, analyze, and test digital circuits that make up the basic building blocks of a computer — such as logic gates, adders, registers, multiplexers, memory units, and processors.

These tools help students, engineers, and researchers to simulate computer hardware before physically building it, reducing time, cost, and errors.

Common CAD Tools Used for Computer Building Blocks:

1. Logisim / Logisim Evolution

Type: Free, educational digital logic simulator.

Use: Designing and testing basic logic circuits (AND, OR, NOT, adders, counters, etc.).

Features:

- Graphical drag-and-drop interface.
- Simulation of circuits in real time.
- Creation of hierarchical circuits (like CPU or memory models).

Application Example:

Simulate a 4-bit ALU, binary adder, or simple CPU design.



2. Multisim

- Type: Professional electronic circuit simulation tool.
- Use: Simulates both analog and digital circuits.
- Features:
 - Supports TTL and CMOS logic families.
 - Realistic timing and voltage behavior.
 - Integration with hardware through NI devices.
- Application Example:
Design and test digital adders, flip-flops, or microprocessor control units.

3. Proteus

- Type: Professional electronic simulation and PCB design suite.
- Use: For simulating microprocessors, microcontrollers, and digital systems.
- Features:
 - Supports simulation of CPU building blocks (ALU, memory, registers).
 - Can simulate embedded systems (Arduino, PIC, ARM).
 - Visual display of data flow and logic states.
- Application Example:
Evaluate how ALU and control signals interact within a CPU model.

4. MATLAB / Simulink

Type: High-level modeling and simulation environment.

Use: For mathematical modeling and hardware-level simulation of computer systems.

Features:

- Simulates data flow between components.
- Can design and analyze digital logic, processors, and control systems.
- Integrates with HDL tools (e.g., VHDL, Verilog).

Application Example:

Simulate the performance of an arithmetic logic unit (ALU) or CPU datapath.

5. ModelSim

Type: HDL (Hardware Description Language) simulation tool.

Use: For simulating VHDL or Verilog-based designs.

Features:

- Used in professional chip and processor design.
- Simulates timing, logic, and behavior at the hardware level.
- Widely used in academic and industrial computer architecture labs.

Application Example:

Simulate the internal operations of CPU components like registers, ALU, and control units.

The Behavior of a Simple Processor Implemented at the Logic Circuit Level

Introduction

- A simple processor is the heart of a computer that performs basic operations like fetching, decoding, and executing instructions.

At the logic circuit level, it is made up of digital components such as logic gates, flip-flops, registers, an ALU (Arithmetic Logic Unit), and a control unit.

These components work together to process binary data (0s and 1s).

How the Processor Works (Step-by-Step)

The processor works in three main steps called the Fetch-Decode-Execute Cycle:

1. Fetch:
 - The Program Counter (PC) sends the address of the next instruction to memory.
 - The instruction is fetched (brought) into the Instruction Register (IR).
2. Decode:
 - The Control Unit (CU) looks at the instruction in binary form and decides what needs to be done (e.g., add, store, load).
3. Execute:
 - The ALU performs the operation (like adding two numbers).
 - The result is stored in a register (like the Accumulator).

After this, the Program Counter moves to the next instruction, and the cycle repeats.

Example

Let's say the processor runs the instruction:

ADD A, B

- 1.Fetch: The instruction “ADD A, B” is taken from memory.
- 2.Decode: The Control Unit sees it is an “ADD” instruction.
- 3.Execute: The ALU adds the contents of A and B.
- 4.The result is stored in the Accumulator (ACC).

All these actions happen using logic circuits — AND, OR, XOR gates, flip-flops, and timing signals.

Formats of Numerical Data from One Format to Another (in Digital Logic and Design)



Introduction :

In Digital Logic and Design, all data inside a computer is represented in binary form (0s and 1s). However, humans often use decimal, octal, or hexadecimal numbers.

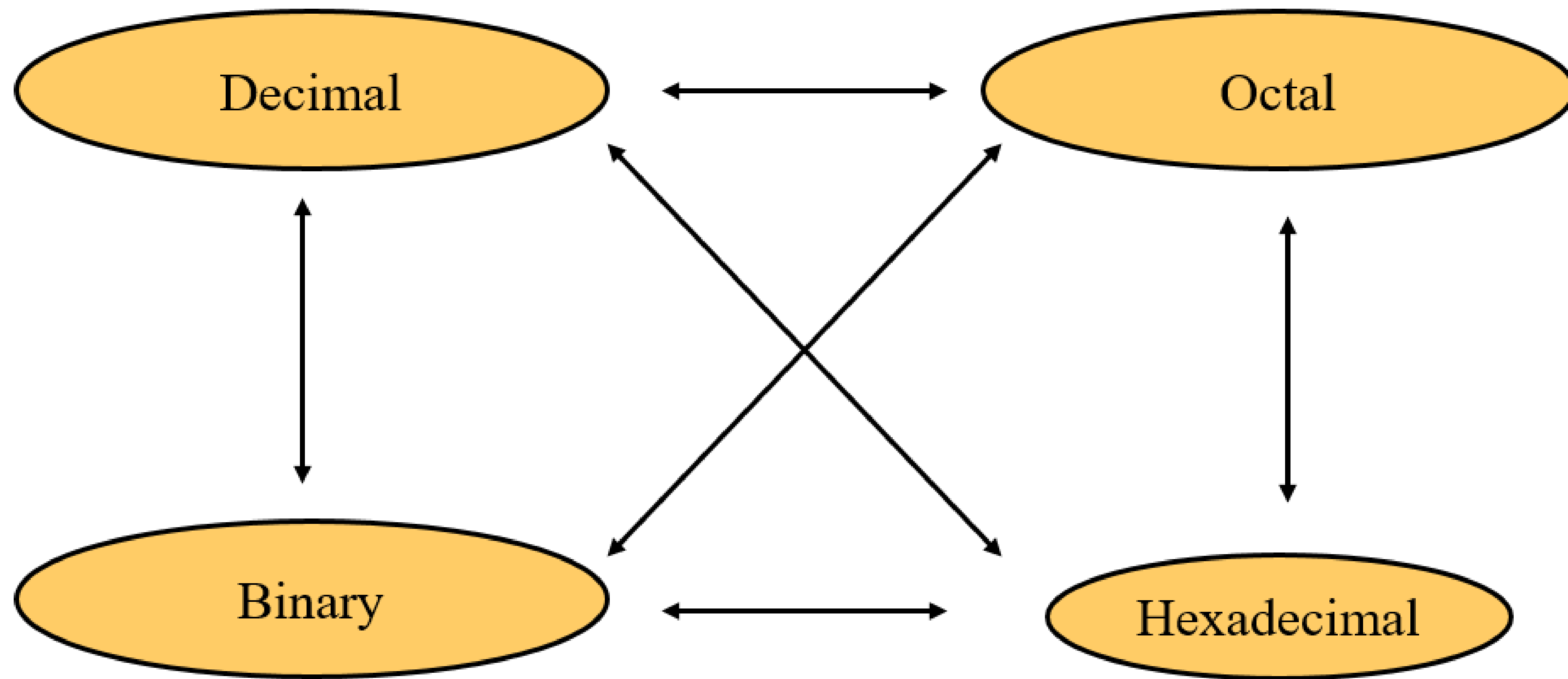
So, it is important to understand how to convert numerical data from one format to another for designing and analyzing digital circuits.

Why Conversion is Important

Different parts of a computer or programming languages may use different number formats.

Conversion helps in:

- Programming (machine code uses binary, humans use decimal/hex).
- Data communication between systems.
- Debugging and error checking.



Decimal to Binary:

Divide the decimal number by 2 repeatedly.
Write down remainders in reverse order.

Example:

Decimal 13 → Binary

$$13 \div 2 = 6 \text{ R}1$$

$$6 \div 2 = 3 \text{ R}0$$

$$3 \div 2 = 1 \text{ R}1$$

$$1 \div 2 = 0 \text{ R}1$$

Binary = 1101

Binary to Decimal

Multiply each binary digit by 2^n (starting from right to left).

Example:

$$\begin{aligned}\text{Binary } 1101 &= (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ &= 8 + 4 + 0 + 1 = 13 \text{ (Decimal)}\end{aligned}$$

Internal Representation of Non-Numeric Data

Introduction

Not all data in a computer is numbers — some are letters, symbols, characters, images, or sounds. Computers, however, can only understand binary (0s and 1s). So, non-numeric data must also be represented in binary form inside the computer. This is called the internal representation of non-numeric data

What is Non-Numeric Data?

Non-numeric data refers to any data that is not a number.

Examples include:

Text/characters → A, B, 1, \$, @

Images → pictures, icons

Audio → sounds, speech

Video → moving pictures

Control data → signals for printers, keyboards, etc.

All these are stored as binary patterns inside the computer.

Representation of non-Numeric Data

DATA TYPE	REPRESENTATION METHOD	BINARY EXAMPLE
TEXT	ASCII / UNICODE	A → 01000001
IMAGE	PIXELS (RGB VALUES)	RED → 11111111 00000000 00000000
SOUND	DIGITAL SAMPLES (AMPLITUDE VALUES)	SAMPLE → 01011010
VIDEO	SEQUENCE OF IMAGE FRAMES	FRAME 1: 101010...

String Processing

What is a String in Digital Systems?

In digital systems, a string is a sequence of binary codes that represent characters such as letters, numbers, or symbols. Each character is stored in binary form using a character encoding standard (like ASCII or Unicode).

Where is String Processing Used in Digital Systems?

String processing is used whenever the digital system needs to:

- Handle text input/output (e.g., from keyboard or display).
- Store or compare names, commands, or instructions.
- Process data streams in communication systems.
- Manage file names, messages, or control strings in microcontrollers or processors.

For example :

String = "HELLO"
→ ASCII codes: 72 69 76 76 79

→ Binary form:

H = 01001000

E = 01000101

L = 01001100

L = 01001100

O = 01001111

So internally, the string "HELLO" is stored as a stream of bits (0s and 1s) in memory.



Decimal (Integer) to Binary Conversion

- Divide the number by the 'Base' (=2)
- Take the remainder (either 0 or 1) as a coefficient
- Take the quotient and repeat the division

Example: **(13)**₁₀

	Quotient	Remainder	Coefficient
13 / 2 =	6	1	$a_0 = \mathbf{1}$
6 / 2 =	3	0	$a_1 = \mathbf{0}$
3 / 2 =	1	1	$a_2 = \mathbf{1}$
1 / 2 =	0	1	$a_3 = \mathbf{1}$

Answer: **(13)**₁₀ = (**a₃ a₂ a₁ a₀**)₂ = (**1101**)₂

MSB LSB

Binary to Decimal Conversion

- Technique
 - Multiply each bit by 2^n , where n is the “weight” of the bit
 - The weight is the position of the bit, starting from 0 on the right
 - Add the results

Example: $101011_2 \Rightarrow 1 \times 2^0 = 1$

$$1 \times 2^1 = 2$$

$$0 \times 2^2 = 0$$

$$1 \times 2^3 = 8$$

$$0 \times 2^4 = 0$$

$$1 \times 2^5 = 32$$

$$43_{10}$$

Octal $(\dots)_8$

octal

Base = 8

OCTAL	BINARY
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Octal $(\dots)_8$

Conversions from octal to binary

$$(7164)_8 = (111\ 001\ 110\ 100)_2$$

Conversions from binary to octal

$$(101\ 101\ 111\ 011\ 000\ 111\ 011)_2 = (5573073)_8$$

Hexadecimal (.....)₁₆

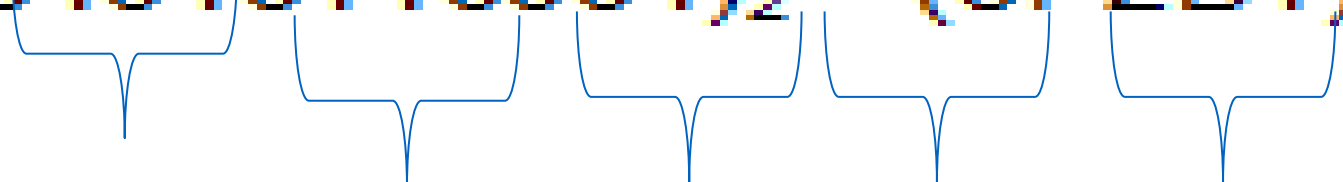
Hexadecimal

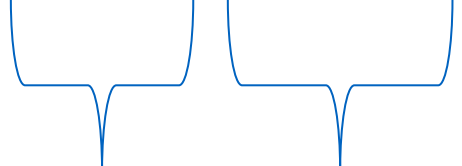
Base = 16

HEXADECIMAL	DECIMAL	BINARY
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Hexadecimal (.....)₁₆

Conversions from binary to Hexadecimal

$$(01011111001010110001)_2 = (5F2B1)_{16}$$


$$(1001100110010100011001)_2 = (266519)_{16}$$


Conversions from Hexadecimal to binary

$$(FA7C9)_{16} = (1111\ 1010\ 0111\ 1100\ 1001)_2$$

	8	4	2	1
9	1	0	0	1
C	1	1	0	0
7	0	1	1	1
A	1	0	1	0
F	1	1	1	1

Binary Operations

♦ Binary Addition

One bit addition:

0	0	1	1
+ 0	+ 1	+ 0	+ 1
-----	-----	-----	-----
0	1	1	2

1 0

carry

2 doesn't exist in binary!

$$(10)_2 = 2 = 1+1$$

$$3 = (11)_2 = (1+1+1)_2$$

Binary Codes

▣ BCD Code

- ◆ A number with k decimal digits will **require 4k bits in BCD**.

- ◆ Decimal **396** is represented in BCD with 12bits

3 9 6
0011 1001 0110

- ▣ Ex: represent 105 using **BCD**

1 0 5
0001 0000 0101

Binary-Coded Decimal (BCD)

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Other Digital Logic Gates

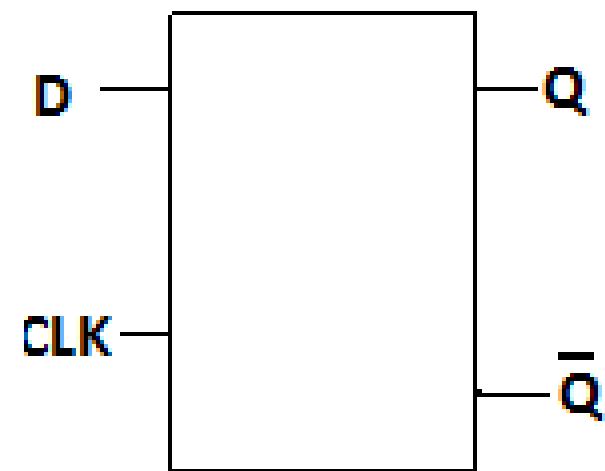
NAND		$F = (xy)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = (x + y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	0
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= (x \oplus y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

Flip-Flops

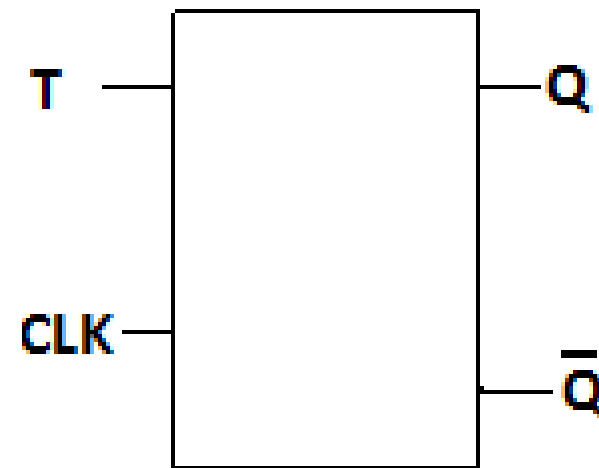
Flip flop: **store 1 bit**

Types:

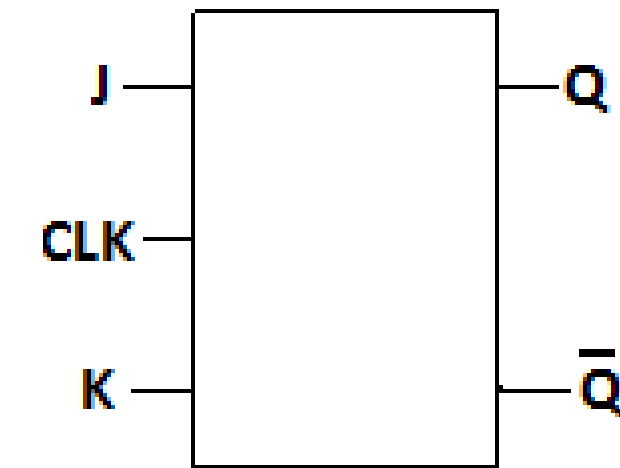
D Flip flop



T Flip flop



JK Flip flop

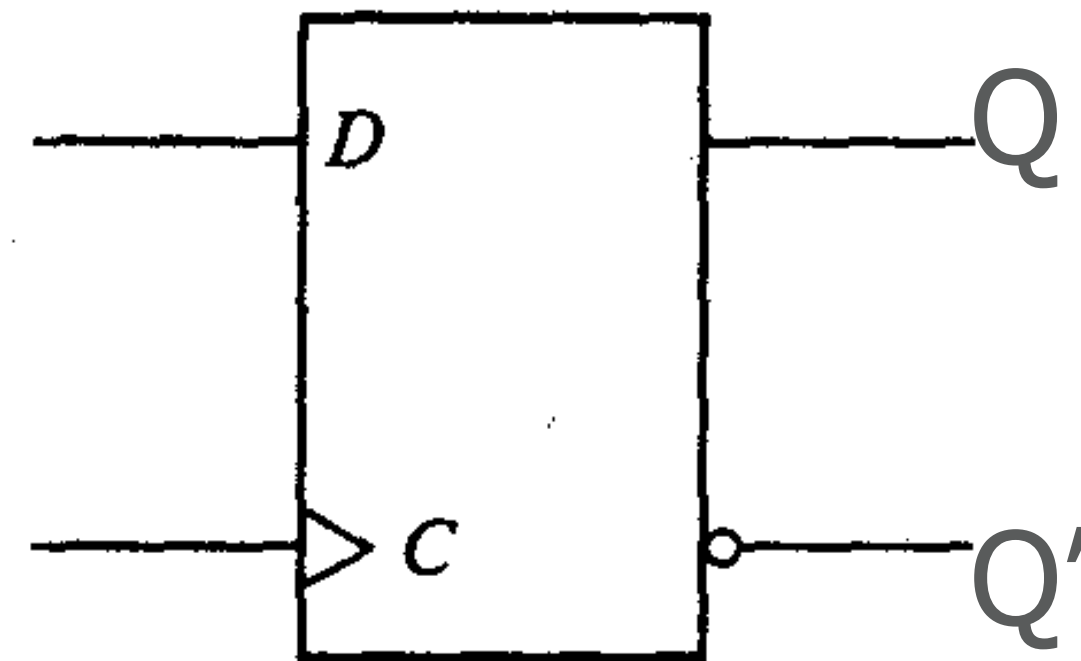


Flip-Flops

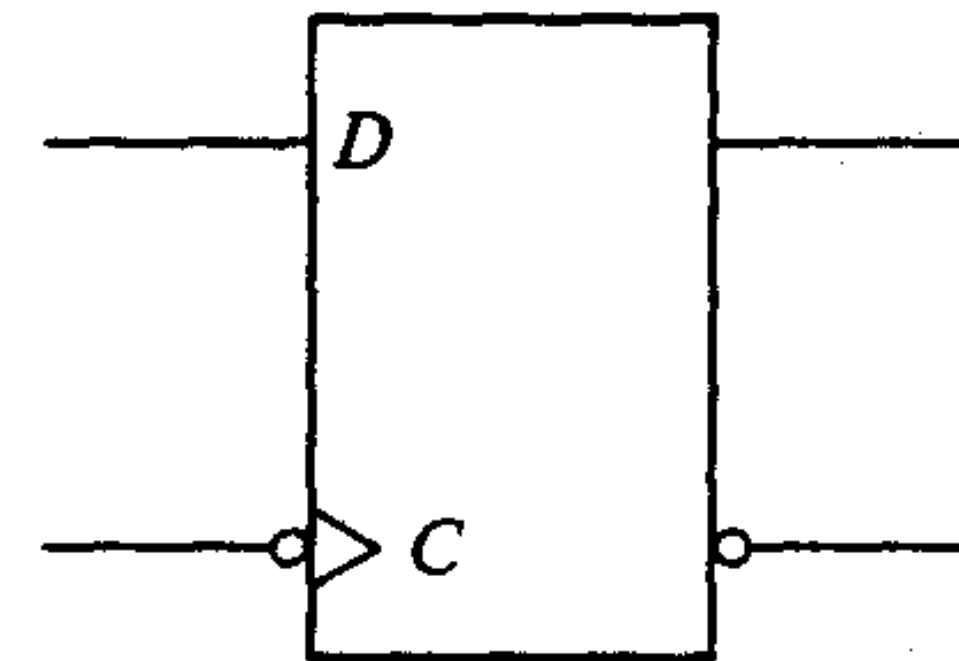
The edge-triggered D flip-flops

The most economical and efficient

Positive-edge and negative-edge



(a) Positive-edge



(a) Negative-edge

$$Q = D$$



State diagram

State transition diagram

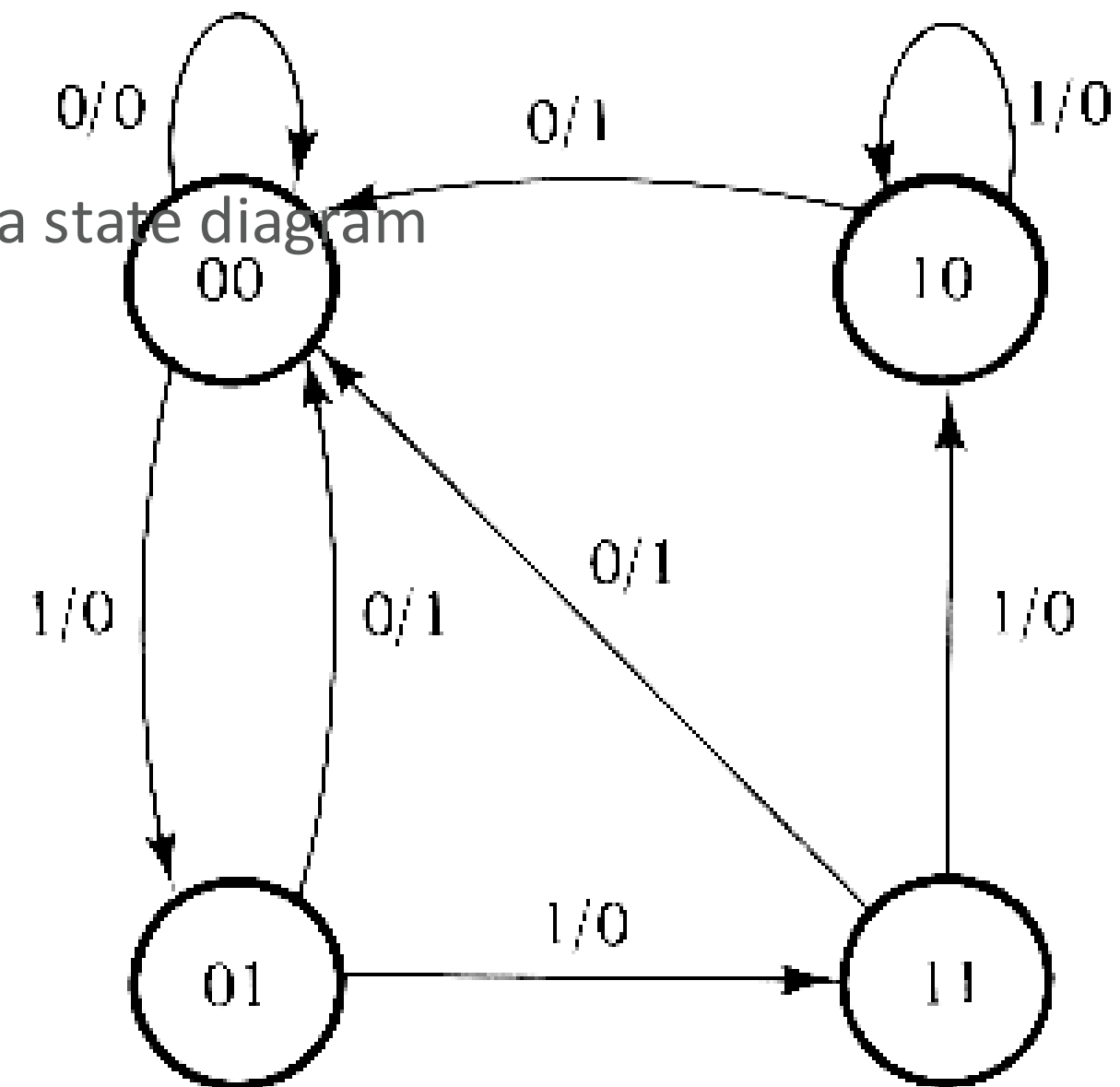
- input A, B $2 \times 2 = 4$

a circle: a state

a directed lines connecting the circles: the transition between the states

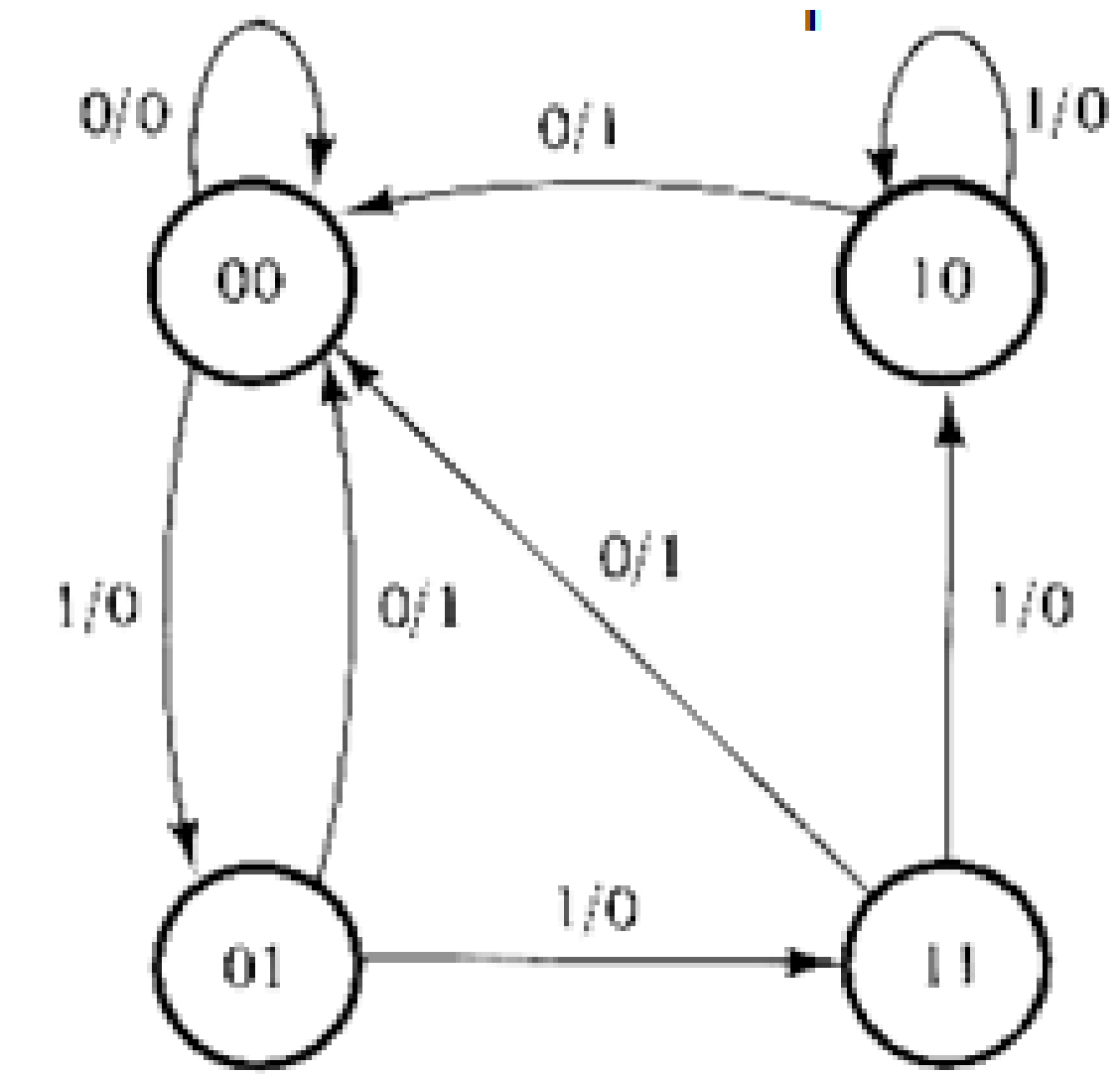
Each directed line is labeled 'inputs/outputs'

a logic diagram $\Leftrightarrow$ a state table $\Leftrightarrow$ a state diagram



State table

State tra = state	Present State		Input	Next State		Output
	A	B		A	B	
	0	0	0	0	0	0
	0	0	1	0	1	0
	0	1	0	0	0	1
	0	1	1	1	1	0
	1	0	0	0	0	1
	1	0	1	1	0	0
	1	1	0	0	0	1
	1	1	1	1	0	0



Present State	Next State		Output	
	$x = 0$		$x = 1$	
	AB	AB	y	y
00	00	01	0	0
01	00	11	1	0
10	00	10	1	0
11	00	10	1	0



Ex: [

Present State		Input	Next State		Output
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

gram

